CLIPOD

A Distraction-Free Offline Music Player

Complete Product Specification & Build Document

Version 1.0 · March 2026 · Open Source / Personal Use

1. Product Vision

Clipod is a small, clip-on, wired audio player that runs Linux. It plays music stored on a microSD card and streams audio from YouTube over Wi-Fi. There is no phone required during use. The phone is used only during setup — to transfer music files and configure Wi-Fi. Once set up, Clipod works completely independently.

The product is philosophically identical to the original iPod: a dedicated device for one thing only, done well. The difference is that instead of syncing from iTunes and a CD library, the user's music comes from YouTube (streamed or downloaded as audio files) or from their own MP3/FLAC collection.

1.1 The problem being solved

Most people use a smartphone to listen to music. A smartphone introduces constant distractions — notifications, messages, social media. For people who run, work out, or commute and want to be fully offline from their phone, there is no good affordable option. The iPod Shuffle was discontinued in 2017. Modern DAPs (digital audio players) cost $150–$500 and are aimed at audiophiles, not athletes or minimalists.

Clipod fills the gap: a sub-$70 device with physical controls, a small OLED screen, a headphone jack, a clip, and nothing else.

1.2 The one-sentence pitch

Core value proposition

Clip it to your pants, plug in your earphones, press play. No phone. No distractions. No subscriptions.

1.3 What Clipod is NOT

- Not a smartphone replacement
- Not a Bluetooth speaker
- Not a streaming device that requires always-on internet
- Not an audiophile DAP competing on sound quality specs
- Not a product that needs a Spotify or Apple Music subscription to function

2. Target Use Cases & Users

2.1 Primary use case — running without a phone

The clearest use case is running. Many runners leave their phone at home intentionally — for safety, weight, focus, or habit. They currently have no good way to listen to music. Clipod clips to a waistband or shorts, the wire runs to earphones, and the device operates entirely standalone. Battery covers a full run (target: 10–14 hours playback from a 600mAh cell at low power draw).

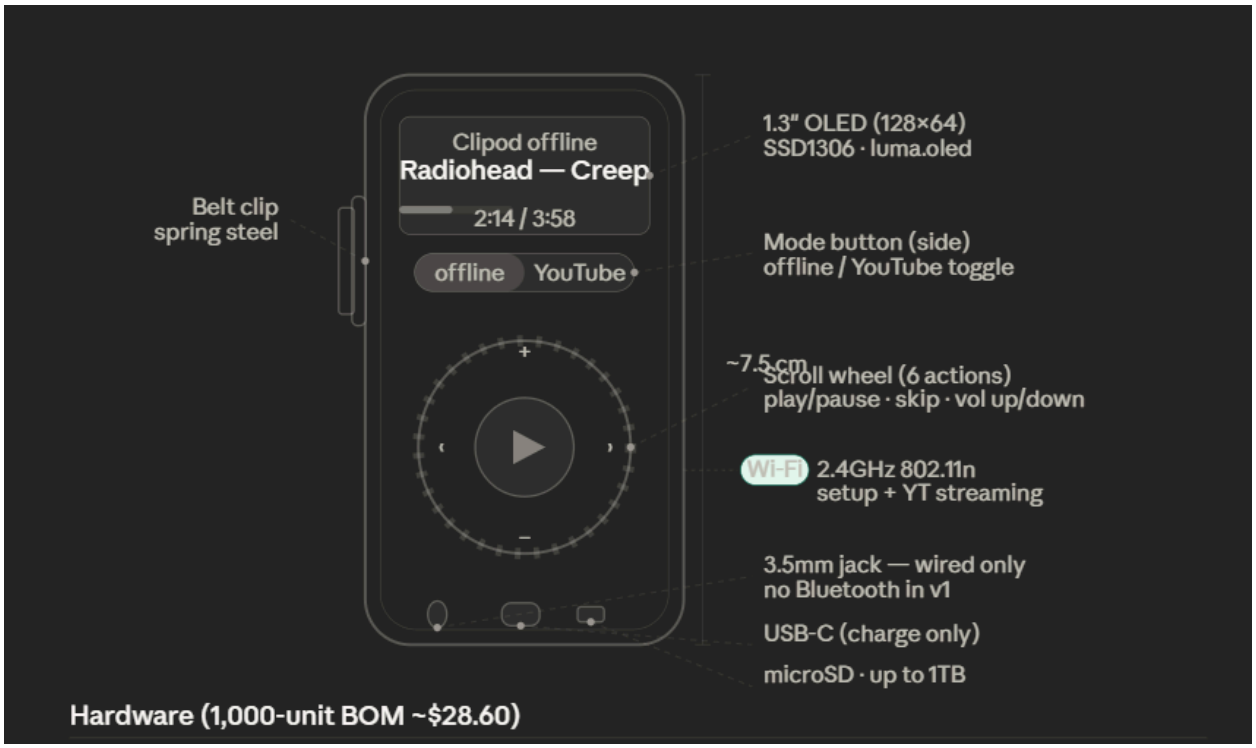
2.2 Secondary use cases

- Gym workouts where pulling out a phone is inconvenient or unsafe
- Commuting where the user wants to ignore notifications
- Work where background music helps focus but phone distractions do not
- Digital minimalism — people intentionally reducing smartphone use
- Children — parents who want kids to have music access without a phone

2.3 User profile

Attribute	Description
Age range	16–40 primarily, no upper limit
Tech comfort	Comfortable installing an app, not necessarily a developer
Music source	YouTube library, personal MP3 collection, or both
Existing gear	Has a smartphone (Android or iPhone) and wired earphones
Willingness to pay	$50–$75 for the device; zero for ongoing subscriptions
Key motivation	Wants music without carrying a phone or dealing with distractions

3. Hardware Design



3.1 Physical form factor

The device should be as close to an iPod Shuffle (4th gen) as possible in size — roughly 45mm × 45mm × 8mm. Slightly larger is acceptable if it improves battery life or assembly. The target weight is under 25g without earphones.

- Body: machined aluminium or injection-moulded ABS plastic (plastic preferred for cost and antenna performance)
- Clip: stainless steel spring clip permanently attached to the rear, strong enough to clip to denim or thick gym shorts
- Colour options (v1): matte black, matte white
- OLED: 1.3-inch SSD1306 or SH1106, 128×64 pixels, white on black, I2C interface
- The OLED shows: track title, artist, playback mode (offline/streaming), battery indicator, Wi-Fi indicator

3.2 Button layout

Six physical buttons. All dome tactile switches (10mm, rated 1 million cycles minimum). Eyes-free operation must be possible by feel alone — buttons are differentiated by position and a raised centre nub on the play/pause button.

Button	Function
Play / Pause (centre, raised nub)	Short press: play/pause. Long press (2s): power on/off
Next (right of centre)	Short press: skip forward. Long press: fast forward
Previous (left of centre)	Short press: skip back or restart track. Long press: rewind
Volume Up (top)	Increase volume. Long press: increase faster
Volume Down (bottom)	Decrease volume. Long press: decrease faster
Mode (rear or side)	Toggle between offline mode and YouTube streaming mode

3.3 Connectivity & ports

- 3.5mm headphone jack: primary audio output. CTIA standard (mic/remote on compatible earphones)
- USB-C (bottom edge): charging only in v1. Data transfer via Wi-Fi, not cable
- microSD slot (side): supports up to 1TB cards. Hot-swappable. User loads music files here
- Wi-Fi: 2.4GHz 802.11n via onboard antenna. Used for YouTube streaming and initial setup only
- No Bluetooth in v1 — adds cost and complexity. Wired-first is the design philosophy

3.4 Recommended SoC options

The SoC (System on Chip) is the brain of the device. It must run Linux, drive an OLED over I2C, decode MP3/FLAC/Opus audio in software, handle Wi-Fi, and read a microSD card. It does not need to be powerful — audio decoding is extremely lightweight.

Chip	Cost (qty 1000)	Notes
Allwinner F1C200s	~$1.80	Very cheap, runs mainline Linux, used in DIY DAPs. Limited RAM (64MB). Sufficient for offline playback. YouTube streaming may be slow.
Allwinner V3s	~$2.50	64MB RAM built-in, ethernet-capable variant. Good balance for the price. Community support exists.
Rockchip RK3128	~$4.00	Quad-core ARM Cortex-A7, 512MB+ RAM possible.

		Comfortable headroom for yt-dlp and streaming. Recommended for v1 if cost allows.
Raspberry Pi Zero 2W	~$15.00	Not for production — too expensive and large. Ideal for software prototyping before custom PCB.

Recommendation

Prototype on Raspberry Pi Zero 2W. Ship v1 on Rockchip RK3128 with 256MB RAM. Move to Allwinner V3s for cost reduction in v2 once software is stable.

3.5 Bill of materials estimate

At 1,000 unit production run. Prices are approximate and will vary by supplier and negotiation.

Component	Estimated cost (USD)
SoC (Rockchip RK3128)	$4.00
RAM (256MB LPDDR3)	$2.50
NAND flash (4GB internal)	$1.50
OLED display (1.3" SSD1306)	$2.20
Wi-Fi module (ESP8285 or similar)	$1.80
Audio codec (PCM5102 or WM8960)	$1.20
3.5mm jack	$0.30
USB-C connector + charging IC (TP4056)	$0.80
microSD slot	$0.40
Battery (600mAh LiPo)	$2.00
Tactile switches x6	$0.60
PCB fabrication + assembly	$6.00

Enclosure (injection mould amortised)	$3.50
Spring clip	$0.80
Miscellaneous (resistors, caps, antenna)	$1.00
TOTAL BOM	~$28.60

At $28.60 BOM, a retail price of $65–$75 is achievable with healthy margin (direct-to-consumer). If sold through a distributor taking 30% cut, retail would need to be $80+.

4. Software Architecture

Philosophy

The software stack is minimal by design. No desktop environment, no GUI framework, no unnecessary services. Everything is purpose-built for audio playback and basic navigation.

4.1 Linux kernel and base system

The device runs a stripped Linux kernel (5.15 LTS recommended for long-term support). Only the drivers actually needed are compiled in:

- I2C driver — for OLED display
- SPI/MMC driver — for microSD card
- GPIO driver — for physical buttons
- ALSA driver — for audio output via the codec chip
- cfg80211 + mac80211 + Wi-Fi chip driver — for wireless
- USB gadget driver — for USB-C charging detection

Base userspace is built with Buildroot — a tool that produces a minimal Linux root filesystem from scratch. Target image size: under 64MB. Boot time target: under 4 seconds from power button to audio playing.

No systemd. Use runit or s6 for init and service supervision — far lighter and faster.

4.2 Audio playback — MPD

MPD (Music Player Daemon) handles all local file playback. It runs as a background service, reads from the microSD mount point, maintains a music library database, and exposes a simple TCP socket protocol for control. It natively supports MP3, FLAC, OGG Vorbis, AAC, WAV, and Opus.

Your button handler daemon connects to MPD's socket and issues commands:

```
# MPD control commands (sent over TCP socket port 6600)
play        # resume playback
pause       # pause
next        # skip to next track
previous    # go to previous track
setvol 75   # set volume to 75%
shuffle     # randomise playlist
status      # get current track info for OLED display
```

MPD is battle-tested, runs on embedded Linux without issues, and uses negligible CPU during playback (audio decoding is hardware-accelerated or extremely cheap on modern ARM chips).

4.3 YouTube audio client

This is the most technically interesting and novel part of Clipod. It is a lightweight daemon written in Python (or C for performance) that provides YouTube audio streaming and search on a device with no browser and a tiny OLED screen.

4.3.1 How YouTube search works

YouTube provides an official Data API v3. Each user gets a free API key from Google Cloud Console (takes 5 minutes, requires a Google account). The free quota is 10,000 units per day. One search query costs 100 units — that is 100 searches per day, more than enough for personal use.

The search request is a standard HTTPS GET:

```
GET https://www.googleapis.com/youtube/v3/search
  ?part=snippet
  &q=radiohead+creep
  &type=video
  &maxResults=10
  &key=YOUR_API_KEY

Response: JSON array of video IDs, titles, channel names, thumbnails
```

The device receives this JSON, parses the titles, and displays them on the OLED as a scrollable list navigated by the Next/Previous buttons.

4.3.2 How audio streaming works (yt-dlp)

yt-dlp is a command-line tool (and Python library) that extracts direct audio stream URLs from YouTube video pages. It does not download the file — it returns a URL pointing to YouTube's CDN that any media player can stream directly.

```
# yt-dlp used as a library in Python
import yt_dlp

def get_audio_url(video_id):
    opts = {
        'format': 'bestaudio[ext=webm]/bestaudio',
        'quiet': True,
        'no_warnings': True,
```

```
    }
    with yt_dlp.YoutubeDL(opts) as ydl:
        info = ydl.extract_info(
            f'https://youtube.com/watch?v={video_id}',
            download=False
        )
        return info['url']  # Direct CDN stream URL

# Feed URL directly to ffmpeg for playback
# ffmpeg -i STREAM_URL -f alsa default
```

ffmpeg (or mpv in audio-only mode) receives the stream URL and pipes the audio to the ALSA audio output. The user hears audio within 2–4 seconds of selection. No file is written to disk.

4.3.3 OLED display and navigation UI

The OLED driver uses the luma.oled Python library (for SSD1306/SH1106) or a custom C implementation using the Linux framebuffer. The display is updated by a single UI daemon that reads state from MPD (for local playback) or from the YouTube client daemon (for streaming).

Display states and what they show:

Screen state	OLED content
Idle / home	Clipod logo + battery % + Wi-Fi status
Local playback	Track title (scrolling if long) + artist + progress bar + volume
YouTube search entry	Search prompt + character selector (alphabet scroll)
YouTube results list	Up to 4 results visible, scrollable, selected item highlighted
YouTube streaming	Video title + channel + buffering indicator + playback progress
Settings menu	Wi-Fi setup / API key entry / storage info / version number

Character input for search uses a simple alphabet-scroll UX: Volume Up/Down scrolls through A–Z + 0–9 + space + backspace, Next selects the highlighted character, Previous deletes last character, Play/Pause confirms and submits the search. Slow to type but functional for short artist/track names.

4.3.4 Full software component map

```
Linux kernel (stripped, ~64MB image)
├── runit (init, service supervision)
├── mpd.service        ← local file playback daemon
```

```
├── button-handler.service ← reads GPIO, sends commands
│   ├── → MPD socket (local playback control)
│   └── → yt-client socket (streaming control)
├── yt-client.service    ← YouTube search + stream daemon
│   ├── YouTube Data API v3 (search)
│   ├── yt-dlp (stream URL extraction)
│   └── ffmpeg (stream → ALSA output)
├── oled-ui.service      ← display renderer
│   ├── reads MPD status
│   ├── reads yt-client status
│   └── writes to I2C OLED via luma.oled
└── setup-server.service ← local HTTP server for initial config
    ├── Wi-Fi credentials entry
    └── YouTube API key entry
    (accessible from phone browser on same network)
```

4.4 Companion phone app

The companion app (Android and iPhone, open source) handles one job: transferring audio files from the user's phone to the Clipod's microSD card over local Wi-Fi. It is not required for the device to function — it is a convenience tool.

4.4.1 How file transfer works

Clipod runs a tiny HTTP server (written in Python using the http.server standard library, under 100 lines of code) on port 8080 when in setup/transfer mode. The companion app discovers it via mDNS (the same protocol that finds Chromecasts and AirPlay devices on a local network) and sends audio files as HTTP POST requests.

```
# Clipod HTTP server (Python, simplified)
from http.server import HTTPServer, BaseHTTPRequestHandler
import os

SD_MUSIC_PATH = '/mnt/sdcard/Music/'

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        filename = self.headers['X-Filename']
        length = int(self.headers['Content-Length'])
        data = self.rfile.read(length)
        with open(os.path.join(SD_MUSIC_PATH, filename), 'wb') as f:
            f.write(data)
        # Trigger MPD library update
        os.system('mpc update')
        self.send_response(200)
```

```
        self.end_headers()

HTTPServer(('0.0.0.0', 8080), Handler).serve_forever()
```

The companion app UI is minimal: a file picker showing audio files on the phone, a Clipod selector (if multiple devices on network), a send button, and a progress indicator. Target: under 500 lines of code total.

4.4.2 Spotify playlist migration tool (optional feature)

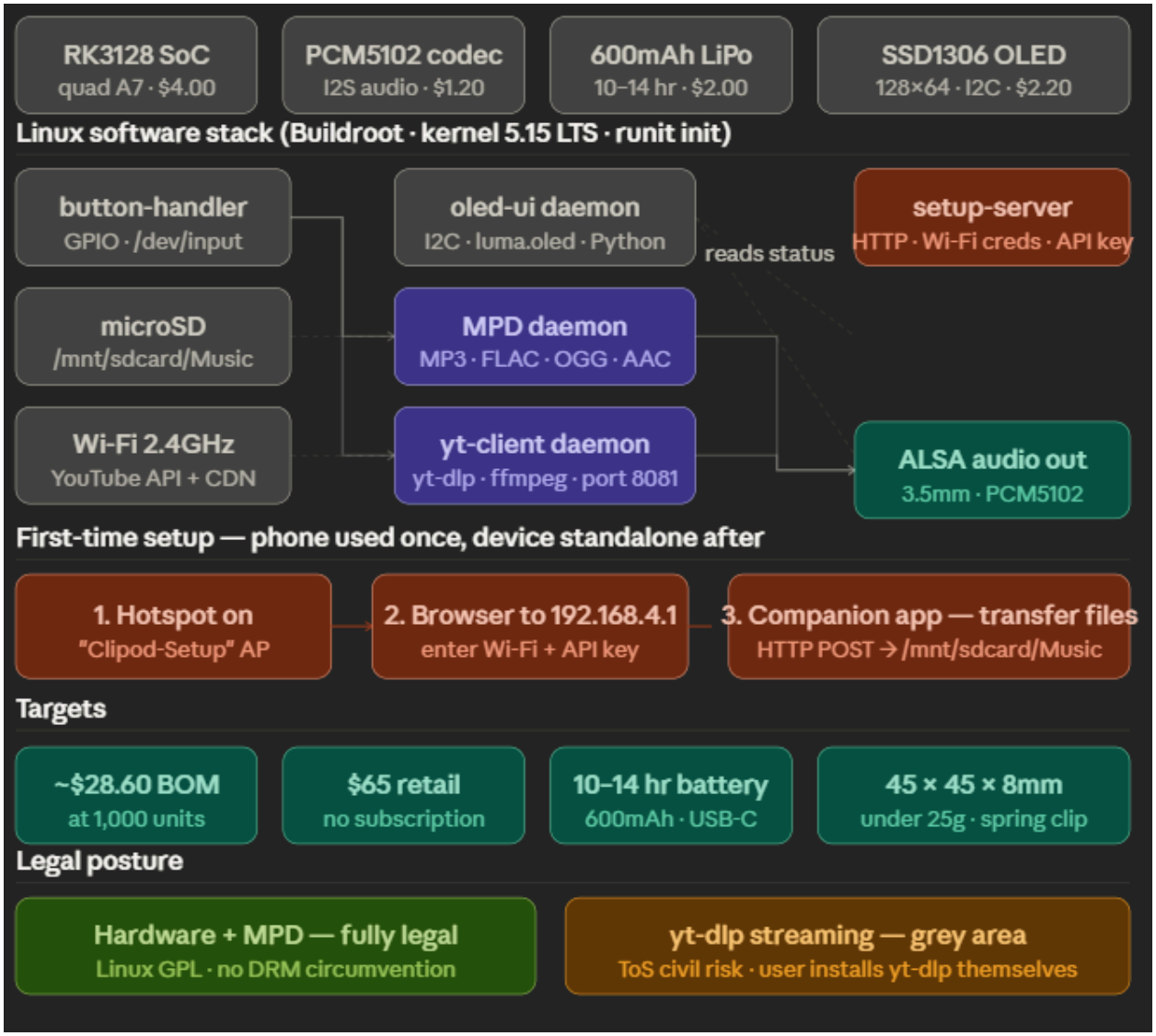
A separate tab in the companion app accepts a Spotify playlist URL (public playlists only, no login required). It uses the Spotify Web API to fetch all track names and artist names, then searches YouTube for each track, and presents a list of matches for the user to confirm. Once confirmed, it adds the YouTube video IDs to a playlist file on the Clipod. The device can then queue and stream those tracks directly from YouTube.

This requires no Spotify account on the device. It only reads public playlist metadata. It is not downloading anything from Spotify — it is reading track names and looking them up on YouTube.

4.5 Setup process (first time)

The total setup time for a non-technical user should be under 10 minutes.

1. Power on Clipod. It creates a temporary Wi-Fi hotspot named 'Clipod-Setup'.
2. User connects phone to 'Clipod-Setup' network.
3. User opens browser on phone and goes to 192.168.4.1 — Clipod's setup page loads.
4. User enters their home Wi-Fi credentials. Clipod connects and gets an IP address.
5. User optionally enters their YouTube Data API key (obtained free from console.cloud.google.com).
6. Setup complete. Clipod restarts into normal mode. Phone reconnects to home Wi-Fi.
7. User opens companion app, discovers Clipod on local network, and transfers music files.
8. Device is now fully configured. Unplug, clip to clothing, plug in earphones, press play.


RK3128 SoC
quad A7 · $4.00
PCM5102 codec
I2S audio · $1.20
600mAh LiPo
10–14 hr · $2.00
SSD1306 OLED
128×64 · I2C · $2.20
Linux software stack (Buildroot · kernel 5.15 LTS · runit init)
button-handler
GPIO · /dev/input
oled-ui daemon
I2C · luma.oled · Python
reads status
setup-server
HTTP · Wi-Fi creds · API key
microSD
/mnt/sdcard/Music
MPD daemon
MP3 · FLAC · OGG · AAC
Wi-Fi 2.4GHz
YouTube API + CDN
yt-client daemon
yt-dlp · ffmpeg · port 8081
ALSA audio out
3.5mm · PCM5102
First-time setup — phone used once, device standalone after
1. Hotspot on
"Clipod-Setup" AP
2. Browser to 192.168.4.1
enter Wi-Fi + API key
3. Companion app — transfer files
HTTP POST → /mnt/sdcard/Music
Targets
~$28.60 BOM
at 1,000 units
$65 retail
no subscription
10–14 hr battery
600mAh · USB-C
45 × 45 × 8mm
under 25g · spring clip
Legal posture
Hardware + MPD — fully legal
Linux GPL · no DRM circumvention
yt-dlp streaming — grey area
ToS civil risk · user installs yt-dlp themselves

5. Market Positioning & Pricing

5.1 Competitive landscape

There is no direct competitor at the $50–$75 price point combining a clip form factor, offline playback, and YouTube streaming. The closest existing products are:

Product	Price	Key gap
iPod Shuffle (discontinued 2017)	$49 used	No screen, iTunes only, no streaming, no longer made
Mighty Vibe	$86	No screen, Spotify/Amazon only, no microSD, expensive
HIFI Walker H2 Touch	$60	Audiophile focus, large size, no clip, not sports-oriented
INNIOASIS G1	$50	Runs Android, poor build quality, no YouTube integration
Garmin Forerunner (watch)	$250+	Wrist computer, different category, expensive

5.2 Pricing strategy

- Device only: $65 recommended retail price
- Device + earphones bundle (with KZ or CCA IEM): $80
- The companion app and all software: free and open source, always
- No subscriptions, no cloud service, no ongoing revenue from users

Revenue model for sustainability: one-time hardware sale. If the project gains traction, a Kickstarter or Indiegogo campaign is the natural first step — this also validates demand before committing to a large production run.

6. Legal Analysis by Region

Important disclaimer

This section reflects general legal understanding as of 2026 and is not formal legal advice. Laws change and vary by jurisdiction. If you commercialise this product, consult a lawyer in your target market.

The legal analysis covers three distinct activities: (A) building a Linux device, (B) streaming YouTube audio, and (C) the companion app that transfers user-owned files.

6.1 Building a Linux device (all regions)

Completely legal everywhere. Linux is open source software licensed under the GPL v2. You are free to use it, modify it, and distribute devices running it, provided you make your kernel source modifications available (GPL requirement). Buildroot is MIT licensed. MPD is GPL licensed. ffmpeg is LGPL/GPL licensed.

GPL compliance requirement

If you sell a device running a modified Linux kernel or GPL software, you must make the source code of those modifications publicly available — for example, on GitHub. This is a legal requirement, not optional. Most hardware companies fulfil this by publishing their board support packages.

6.2 YouTube streaming (yt-dlp usage)

This is the most legally nuanced area and varies by jurisdiction.

6.2.1 YouTube's Terms of Service

YouTube's Terms of Service (Section 5.B) prohibit downloading or streaming content outside of YouTube's official playback mechanisms without explicit permission. This applies globally because it is a contractual term, not law.

However: ToS violations are a civil matter between YouTube and the user. YouTube can terminate the user's account and potentially sue for breach of contract. In practice, YouTube has never sued an individual user for personal audio streaming using a tool like yt-dlp. They send DMCA notices to the tool developers (yt-dlp receives these regularly and continues to operate), but users themselves are not targeted.

The key distinction: yt-dlp is being used here to stream audio in real time, not to create permanent copies of copyrighted material. The audio is never written to disk. This is meaningfully different from downloading and distributing music files.

6.2.2 United States

Under US copyright law (17 U.S.C. § 106), the copyright holder controls reproduction and distribution. Temporary RAM copies made during streaming are generally considered fair use or are covered by the AHRA (Audio Home Recording Act). Courts have not definitively ruled on real-time audio streaming via unofficial clients.

The DMCA (Digital Millennium Copyright Act, § 1201) prohibits circumventing technological protection measures. YouTube's stream URL extraction could theoretically be construed as circumventing a protection measure, though legal scholars disagree on whether stream URL extraction qualifies. No user has been prosecuted under § 1201 for personal streaming.

US risk level

Low for personal use. The open source, non-commercial nature of the software significantly reduces risk. YouTube is far more likely to technically block the tool than pursue legal action against users.

6.2.3 European Union

The EU Copyright Directive (Article 17, formerly Article 13) primarily targets large platforms hosting user-uploaded content, not end users or tools. The EU's InfoSoc Directive (2001/29/EC) permits temporary reproductions 'which are transient or incidental and an integral part of a technological process' — real-time streaming fits this description.

Germany has historically been the most aggressive EU jurisdiction on copyright, but cases have targeted file-sharing networks and permanent copies, not personal streaming clients. The UK (post-Brexit) follows similar principles under the CDPA 1988.

EU risk level

Low to moderate. France and Germany are stricter jurisdictions. The non-commercial, personal-use, open-source nature is mitigating. Avoid commercial distribution of the software in Germany without legal advice.

6.2.4 India

India's Copyright Act 1957 (amended 2012) includes a fair dealing exception for private use. Streaming audio from YouTube for personal listening falls comfortably within fair dealing. India has no equivalent to the DMCA's anti-circumvention provisions of comparable strength.

India risk level

Very low. Personal streaming for private use is well within fair dealing provisions.

6.2.5 Australia, Canada, New Zealand

Australia's Copyright Act 1968 includes time-shifting and format-shifting provisions for personal use. Canada's Copyright Modernization Act 2012 explicitly permits reproduction for private use. New Zealand follows similar principles. Real-time personal streaming is extremely unlikely to be actionable in any of these jurisdictions.

ANZ / Canada risk level

Very low.

6.2.6 How to minimise legal exposure regardless of region

- Distribute the companion app as open source under MIT or GPL license
- Do not distribute yt-dlp as a binary bundled with the device firmware — require users to install it themselves during setup (one command: pip install yt-dlp)
- Add a clear disclaimer in the setup process: 'This tool streams audio from YouTube in real time. Ensure your use complies with YouTube's Terms of Service and local copyright law.'
- Do not market the product as a 'YouTube downloader' or 'music ripper' — market it as a streaming audio player
- Do not enable permanent saving of YouTube audio streams to disk in the default firmware — offline mode is for user-owned files only

6.3 The companion file transfer app

Completely legal. It is a Wi-Fi file transfer tool — functionally identical to AirDrop, Send Anywhere, or connecting a USB drive. What the user transfers is their responsibility. Your app does not acquire, download, or source any content. It only moves files the user already has on their phone to the device's SD card.

6.4 Hardware export controls

If manufacturing in China (as most consumer electronics are), note that devices with Wi-Fi chips may be subject to export control regulations (EAR in the US, similar frameworks in EU). These are manufacturer and exporter responsibilities, not typically end-user concerns. A product with Wi-Fi operating at 2.4GHz requires FCC certification (US), CE marking (EU), and BIS certification (India) before commercial sale. Budget $5,000–$15,000 for certification depending on jurisdiction.

7. Development Roadmap

Phase 0 — Software prototype (1–2 months)

Use a Raspberry Pi Zero 2W. Build and test all software before touching custom hardware. This is the most important phase.

- Get MPD playing local files from a USB drive
- Get yt-dlp + ffmpeg streaming YouTube audio to a USB audio adapter
- Build the basic OLED UI using a cheap SSD1306 breakout board
- Wire up 6 tactile buttons to GPIO pins and write the button handler
- Build the HTTP file transfer server
- Integrate everything into a working system on the Pi

Phase 1 — Custom hardware prototype (2–4 months)

- Design PCB in KiCad (free, open source)
- Order prototype PCBs from JLCPCB or PCBWay (under $50 for 5 boards)
- Source components from LCSC or Mouser
- Hand-assemble and test
- Iterate until hardware is stable

Phase 2 — Beta and validation (1–2 months)

- Give 20–50 devices to real users (runners, gym-goers, commuters)
- Collect feedback on physical UX, battery life, Wi-Fi reliability, OLED readability
- Iterate on enclosure design and button feel

Phase 3 — Production run

- Crowdfund via Kickstarter or Indiegogo to validate demand and raise manufacturing capital
- Minimum viable production run: 500 units
- Manufacturing partner: contract manufacturer in Shenzhen (PCBWay, Seeed Studio, or a Huaqiangbei factory contact)
- Certifications: FCC (US), CE (EU) — required before commercial sale

8. Open Source Strategy

All software — the Linux image build system, the YouTube client daemon, the OLED UI, the button handler, the companion app, and the HTTP transfer server — is published on GitHub under the MIT license (or GPL where required by dependencies).

The hardware design — KiCad schematics, PCB layout, enclosure CAD files — is published under CERN-OHL-P (permissive open hardware license).

This is not just an ethical choice. It is a practical one. Open source means:

- Users trust the device is not doing anything hidden with their data or YouTube access
- The community can port it to other hardware, fix bugs, and add features
- GPL compliance is automatically satisfied
- The project has credibility with technically sophisticated early adopters
- Legal exposure is minimised — you are a platform, not a piracy service

The GitHub repository structure:

```
clipod/
├── firmware/        ← Buildroot config + kernel patches
│   ├── mpd-config/
│   ├── yt-client/   ← YouTube streaming daemon (Python)
│   ├── button-handler/
│   ├── oled-ui/
│   └── setup-server/
├── companion-app/   ← Android + iOS file transfer app
│   └── migration/   ← Spotify playlist → YouTube converter
├── hardware/        ← KiCad PCB + enclosure CAD
└── docs/            ← Setup guides, API key instructions
```

9. Summary — What You Are Building

Clipod is a clip-on Linux audio player with a 1.3-inch OLED, six physical buttons, a 3.5mm headphone jack, a microSD slot, and Wi-Fi. It plays user-owned music files from the SD card and streams audio from YouTube over Wi-Fi. It requires no phone during use. It has no subscriptions. It costs around $65 at retail.

The software is fully open source. The YouTube streaming is handled by yt-dlp and ffmpeg, operated by the user for personal use. The companion app is a file transfer tool. No audio is permanently downloaded from YouTube in the default firmware.

The legal risk is low because the product is open source, non-commercial in its software, and positions YouTube streaming as a personal-use client rather than a piracy tool. The hardware sale is the business.

The development path starts with a Raspberry Pi Zero 2W running all the software, then moves to a custom PCB, then to a crowdfunded production run.

The one thing to build first

Get MPD + yt-dlp + ffmpeg + an SSD1306 OLED + 6 GPIO buttons working on a Raspberry Pi Zero 2W. Everything else follows from that working prototype.